

Charte de Codage – Firmware UEFI Microkernel

1. Objectif de la charte

Cette charte propose des conventions de nommage et de structuration pour le développement d'un firmware microkernel UEFI compliant, basé sur Coreboot, lisible et maintenable. Elle permet également de séparer l'API publique UEFI du code interne.

2. Conventions de nommage

Pour le code interne, il est recommandé d'adopter le style Coreboot/Linux : - Fonctions : snake_case, ex: memory_allocate_pool() - Variables : snake_case, ex: boot_order_index - Structs : explicites, ex: struct fw_handle - Types standards C : int, void*, bool, uint32_t, size_t - Minimiser les macros - Alignement horizontal des paramètres

Exemples :

```
// Code interne
static efi_status_t install_protocol(struct fw_handle *handle,
struct fw_protocol *protocol,
void *interface);

// Struct interne
struct fw_handle {
struct fw_protocol_entry *protocols;
};
```

3. Interfaces publiques UEFI

Pour toute API publique exposée conformément à la spécification UEFI, il est impératif : - De conserver les noms exacts imposés par la spec (ex: EFI_STATUS, EFI_HANDLE, InstallProtocolInterface) - De conserver les structures imposées par la spec - D'utiliser les typedefs et conventions de la spec uniquement dans la couche API

Exemple :

```
// API publique (conforme UEFI spec)
EFI_STATUS
InstallProtocolInterface(EFI_HANDLE *handle,
EFI_GUID *protocol,
EFI_INTERFACE_TYPE interface_type,
VOID *interface);
```

4. Séparation API publique et code interne

Stratégie recommandée : 1. Définir une couche API publique (ex: include/uefi_spec.h) qui expose toutes les fonctions et structures imposées par la spec. 2. Implémenter la logique interne dans le code interne, en style Coreboot/Linux, avec typedefs et conventions propres. 3. Mapper les types spec vers les types internes, ex: - EFI_STATUS (public) → efi_status_t (interne) - Chaque fonction API appelle une fonction interne et convertit le type si nécessaire.

Exemple de mapping :

```
// Définition interne
typedef uint64_t efi_status_t;

// Fonction interne
static efi_status_t install_protocol(struct fw_handle *handle,
struct fw_protocol *protocol,
void *interface)
{
// implémentation interne
return 0;
}

// Fonction API publique
EFI_STATUS
InstallProtocolInterface(EFI_HANDLE *handle,
EFI_GUID *protocol,
EFI_INTERFACE_TYPE interface_type,
VOID *interface)
{
efi_status_t status = install_protocol((struct fw_handle *)handle,
(struct fw_protocol *)protocol,
interface);
return (EFI_STATUS)status;
}
```

5. Résumé de la convention

- API publique : respecter nom et structure UEFI - Code interne : style Coreboot/Linux, snake_case, struct explicites, types C standards - Séparer clairement couche interne et couche API - Faire le mapping des types (EFI_STATUS ↔ efi_status_t) à la frontière - Maintenir lisibilité et maintenabilité